

Trading Education Platform - Technical Documentation

Executive Summary

A complete trading education ecosystem combining paper trading, behavioral coaching, and financial literacy. Think: **Zerodha** + **TradingView** + **Duolingo** for trading.

Core Value Proposition:

- Trade with fake money, real market mechanics
 - Behavioral feedback and risk education
 - Professional-grade tools for learning
-

1. System Architecture

1.1 Technology Stack

Frontend:

- Framework: Next.js 14+ (App Router)
- State Management: React Context / Zustand
- UI Components: Tailwind CSS + shadcn/ui
- Charts: TradingView Lightweight Charts / Recharts
- Real-time: WebSocket / Server-Sent Events

Backend:

- Framework: Spring Boot 3.x
- Language: Java 17+
- Architecture: Modular Monolith → Microservices (future)
- API: RESTful + WebSocket

Database:

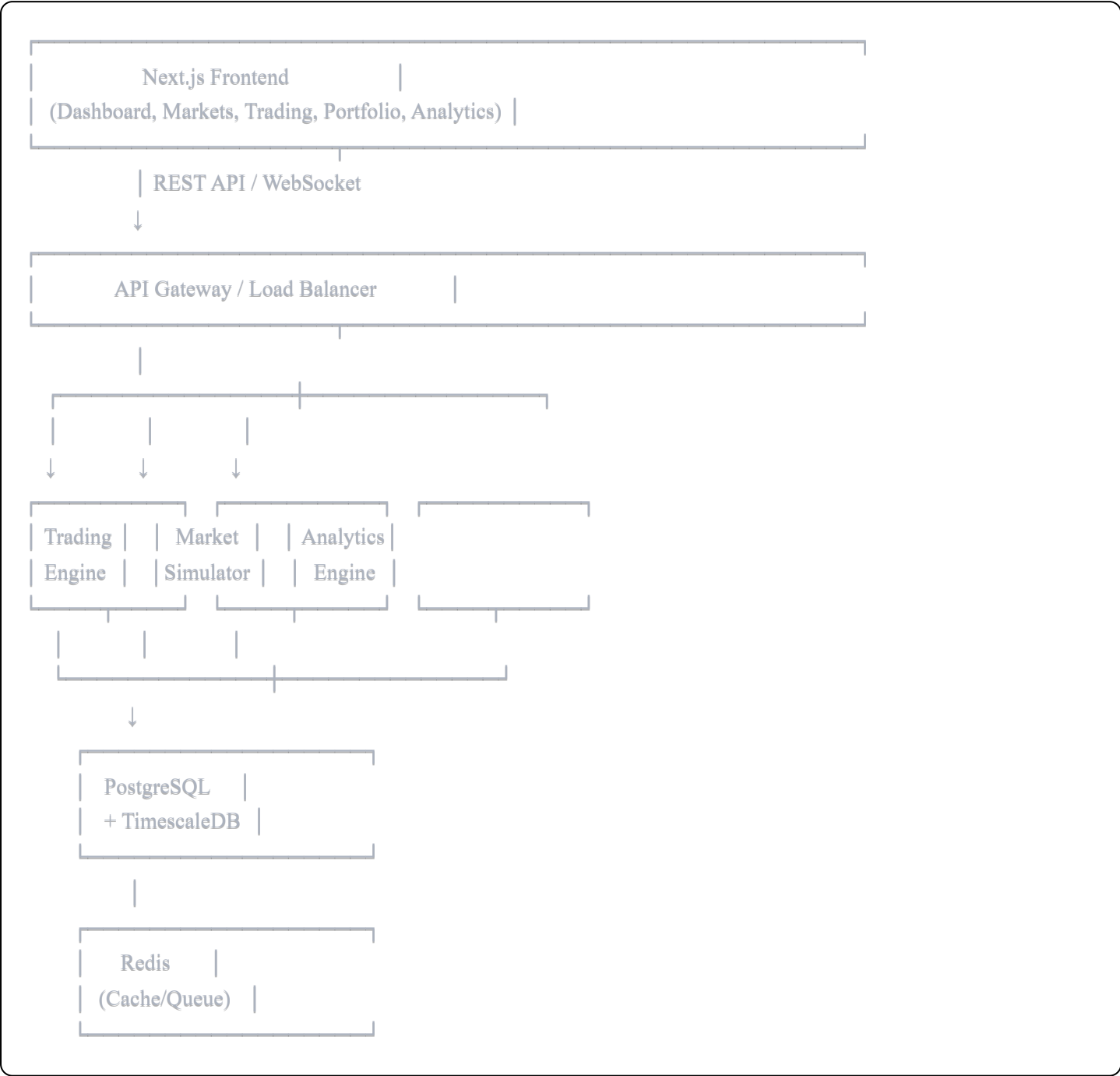
- Primary: PostgreSQL 15+
- Cache: Redis
- Time-Series: PostgreSQL with TimescaleDB extension

Infrastructure:

- Container: Docker

- Orchestration: Kubernetes (production)
- CI/CD: GitHub Actions
- Monitoring: Prometheus + Grafana

1.2 High-Level Architecture Diagram



2. Module Architecture

2.1 MODULE 1 — Core Trading Engine

Purpose: Handle all trading operations, order execution, and portfolio management.

Components:

Services:

- `TradingEngineService` - Core trading logic
- `OrderMatchingService` - Match buy/sell orders
- `PositionService` - Track open positions
- `PortfolioService` - Calculate holdings and valuations

Database Schema:

```
sql
```

-- Orders table

```
orders (  
  id UUID PRIMARY KEY,  
  user_id UUID NOT NULL,  
  asset_id UUID NOT NULL,  
  order_type VARCHAR(10), -- MARKET, LIMIT  
  side VARCHAR(4), -- BUY, SELL  
  quantity INTEGER,  
  price DECIMAL(15,2),  
  status VARCHAR(20), -- PENDING, EXECUTED, CANCELLED  
  created_at TIMESTAMP,  
  executed_at TIMESTAMP  
)
```

-- Trades table

```
trades (  
  id UUID PRIMARY KEY,  
  order_id UUID,  
  user_id UUID,  
  asset_id UUID,  
  quantity INTEGER,  
  price DECIMAL(15,2),  
  side VARCHAR(4),  
  commission DECIMAL(10,2),  
  executed_at TIMESTAMP  
)
```

-- Positions table

```
positions (  
  id UUID PRIMARY KEY,  
  user_id UUID,  
  asset_id UUID,  
  quantity INTEGER,  
  avg_price DECIMAL(15,2),  
  current_price DECIMAL(15,2),  
  unrealized_pnl DECIMAL(15,2),  
  updated_at TIMESTAMP  
)
```

-- Holdings table

```
holdings (  
  id UUID PRIMARY KEY,  
  user_id UUID,  
  asset_id UUID,  
  quantity INTEGER,  
  avg_buy_price DECIMAL(15,2),
```

```
invested_amount DECIMAL(15,2)
)
```

API Endpoints:

```
POST /api/orders    - Place order
GET  /api/orders    - Get user orders
DELETE /api/orders/:id - Cancel order
GET  /api/positions - Get open positions
GET  /api/holdings  - Get portfolio holdings
GET  /api/portfolio/value - Get total portfolio value
```

2.2 MODULE 2 — Market Simulation Engine

Purpose: Generate realistic market data with 50-100 fake companies using advanced mathematical models.

Company Personality Model:

Each company has unique characteristics:

- **Base Price:** Starting price point
- **Volatility:** Price movement intensity (σ)
- **Drift:** Growth/decline trend (μ)
- **Sector:** Industry classification
- **Risk Score:** 1-10 risk rating
- **Market Cap:** Small/Mid/Large cap

Database Schema:

```
sql
```

```

-- Assets table
assets (
  id UUID PRIMARY KEY,
  symbol VARCHAR(10) UNIQUE,
  name VARCHAR(100),
  sector VARCHAR(50),
  base_price DECIMAL(15,2),
  current_price DECIMAL(15,2),
  volatility DECIMAL(5,4), --  $\sigma$  (0.01 to 0.50)
  drift DECIMAL(5,4), --  $\mu$  (-0.02 to 0.05)
  risk_score INTEGER, -- 1-10
  market_cap VARCHAR(20), -- SMALL, MID, LARGE
  description TEXT,
  created_at TIMESTAMP
)

-- Price history (time-series)
prices (
  id BIGSERIAL PRIMARY KEY,
  asset_id UUID NOT NULL,
  price DECIMAL(15,2),
  volume INTEGER,
  timestamp TIMESTAMP,
  -- TimescaleDB hypertable
  INDEX idx_asset_time (asset_id, timestamp DESC)
)

-- Sectors table
sectors (
  id UUID PRIMARY KEY,
  name VARCHAR(50),
  sector_factor DECIMAL(5,4), -- Daily sector movement
  correlation JSONB -- {"TECH": 0.7, "BANK": -0.2}
)

-- Events table
events (
  id UUID PRIMARY KEY,
  asset_id UUID,
  event_type VARCHAR(50), -- EARNINGS, PRODUCT_LAUNCH, SCANDAL
  impact DECIMAL(5,4), -- -0.20 to 0.30
  description TEXT,
  occurred_at TIMESTAMP
)

```

Price Generation Algorithm:

Step 1: Historical Data Generation (Geometric Brownian Motion)

java

```
public List<PricePoint> generateHistory(Asset asset, int days) {
    List<PricePoint> history = new ArrayList<>();
    double price = asset.getBasePrice();
    LocalDateTime date = LocalDateTime.now().minusDays(days);

    for (int i = 0; i < days; i++) {
        // GBM formula:  $dS = \mu S dt + \sigma S dW$ 
        double drift = asset.getDrift();
        double volatility = asset.getVolatility();
        double randomShock = random.nextGaussian(); // Standard normal

        double change = drift + (volatility * randomShock);
        price = price * Math.exp(change);

        history.add(new PricePoint(date.plusDays(i), price));
    }

    return history;
}
```

Step 2: Live Price Updates (Multi-Factor Model)

java

```

public double calculateNextPrice(Asset asset) {
    double currentPrice = asset.getCurrentPrice();

    // Factor 1: Random noise
    double noise = random.nextGaussian() * asset.getVolatility();

    // Factor 2: Drift (trend)
    double drift = asset.getDrift();

    // Factor 3: Sector influence
    double sectorFactor = getSectorFactor(asset.getSector());

    // Factor 4: Market-wide movement
    double marketFactor = getMarketFactor();

    // Factor 5: Mean reversion
    double meanReversion = calculateMeanReversion(asset);

    // Factor 6: Event impact (if any)
    double eventImpact = getActiveEventImpact(asset.getId());

    // Combine all factors
    double totalChange = noise + drift + sectorFactor
        + marketFactor + meanReversion + eventImpact;

    // Apply change
    double newPrice = currentPrice * (1 + totalChange);

    // Price bounds (prevent negative/extreme values)
    return Math.max(newPrice, 0.01);
}

```

Mean Reversion Formula:

```

java

private double calculateMeanReversion(Asset asset) {
    double currentPrice = asset.getCurrentPrice();
    double basePrice = asset.getBasePrice();
    double deviation = (currentPrice - basePrice) / basePrice;

    // Pull back to base price (5% reversion speed)
    return -deviation * 0.05;
}

```

Company Distribution:

Type	Count	Volatility	Drift	Example
Stable Banks	20	0.01-0.05	0.002	HDFC, SBI
Tech Giants	25	0.10-0.20	0.020	TechCorp
Mid-caps	30	0.15-0.30	0.010	GrowthCo
Volatile	15	0.30-0.50	0.000	MemeStock
Crypto-like	10	0.40-0.80	-0.010	VolaCoin

Sector Correlation Implementation:

```
java

// If TECH sector moves +2%, all tech stocks get boost
public double getSectorFactor(String sector) {
    Sector sectorData = sectorRepository.findByName(sector);

    // Update sector factor daily
    if (shouldUpdateSector()) {
        double sectorMove = random.nextGaussian() * 0.02;
        sectorData.setSectorFactor(sectorMove);

        // Correlate related sectors
        applyCorrelation(sectorData);
    }

    return sectorData.getSectorFactor();
}
```

Event System:

```
java
```

```

@Scheduled(cron = "0 0 9 * * *") // Daily at 9 AM
public void generateRandomEvents() {
    List<Asset> assets = assetRepository.findAll();

    for (Asset asset : assets) {
        // 1% chance per day per asset
        if (random.nextDouble() < 0.01) {
            Event event = createRandomEvent(asset);
            eventRepository.save(event);

            // Immediate price impact
            applyEventImpact(asset, event);
        }
    }
}

private Event createRandomEvent(Asset asset) {
    String[] eventTypes = {"EARNINGS_BEAT", "PRODUCT_LAUNCH",
        "SCANDAL", "ACQUISITION", "REGULATION"};

    String type = eventTypes[random.nextInt(eventTypes.length)];
    double impact = random.nextGaussian() * 0.15; // -15% to +15%

    return Event.builder()
        .assetId(asset.getId())
        .eventType(type)
        .impact(impact)
        .description(generateDescription(type, asset))
        .occurredAt(LocalDateTime.now())
        .build();
}

```

Services:

- `PriceSimulatorService` - Real-time price updates
- `HistoryGeneratorService` - Generate historical charts
- `MarketScheduler` - Scheduled price updates
- `SectorEngine` - Manage sector correlations
- `EventEngine` - Generate market events

2.3 MODULE 3 — Wallet & Credits System

Purpose: Virtual currency management and monetization.

Database Schema:

```
sql

wallets (
  id UUID PRIMARY KEY,
  user_id UUID UNIQUE,
  balance DECIMAL(15,2),
  credits INTEGER,
  updated_at TIMESTAMP
)

transactions (
  id UUID PRIMARY KEY,
  user_id UUID,
  type VARCHAR(20), -- TRADE_DEBIT, CREDIT_PURCHASE, REFUND
  amount DECIMAL(15,2),
  balance_after DECIMAL(15,2),
  description TEXT,
  created_at TIMESTAMP
)

credit_packages (
  id UUID PRIMARY KEY,
  name VARCHAR(50),
  credits INTEGER,
  price DECIMAL(10,2),
  bonus_credits INTEGER
)
```

Business Logic:

- Initial balance: ₹100,000 virtual money
- Trade execution: Deduct amount + simulated brokerage
- Credits: Optional purchases for premium features
- Transaction ledger: Audit trail of all movements



2.4 MODULE 4 — Analytics & Insights Engine

Purpose: Performance tracking, behavioral analysis, and reporting.

Database Schema:

```
sql
```

```
analytics_snapshots (  
  id UUID PRIMARY KEY,  
  user_id UUID,  
  date DATE,  
  total_trades INTEGER,  
  win_rate DECIMAL(5,2),  
  avg_profit DECIMAL(15,2),  
  avg_loss DECIMAL(15,2),  
  max_drawdown DECIMAL(15,2),  
  portfolio_value DECIMAL(15,2),  
  risk_score INTEGER  
)  
  
user_metrics (  
  id UUID PRIMARY KEY,  
  user_id UUID,  
  metric_name VARCHAR(50),  
  metric_value DECIMAL(15,4),  
  calculated_at TIMESTAMP  
)
```

Key Metrics Calculated:

1. Win Rate:

```
java  
  
public double calculateWinRate(UUID userId) {  
    List<Trade> trades = tradeRepository.findByUserId(userId);  
    long wins = trades.stream()  
        .filter(t -> t.getPnl() > 0)  
        .count();  
    return (double) wins / trades.size() * 100;  
}
```

2. Max Drawdown:

```
java
```

```

public double calculateMaxDrawdown(UUID userId) {
    List<Double> portfolioValues = getHistoricalValues(userId);
    double peak = portfolioValues.get(0);
    double maxDrawdown = 0;

    for (double value : portfolioValues) {
        if (value > peak) peak = value;
        double drawdown = (peak - value) / peak;
        maxDrawdown = Math.max(maxDrawdown, drawdown);
    }

    return maxDrawdown * 100; // Percentage
}

```

3. Sharpe Ratio (Advanced):

```

java

public double calculateSharpeRatio(UUID userId) {
    List<Double> returns = getDailyReturns(userId);
    double avgReturn = returns.stream()
        .mapToDouble(Double::doubleValue)
        .average()
        .orElse(0);
    double stdDev = calculateStdDev(returns);

    return (avgReturn - 0.04) / stdDev; // Assuming 4% risk-free rate
}

```

2.5 MODULE 5 — Trade Recommendation & Behavioral Coach

Purpose: Intelligent insights and behavioral nudges (NOT prediction).

Architecture:

User Trades → Data Collection → Rule Engine → Insight Generation → Frontend Display

Recommendation Types:

1. Risk-Based Recommendations

```

java

```

@Service

```
public class RiskRecommendationEngine {

    public List<Insight> analyzeRisk(UUID userId) {
        List<Insight> insights = new ArrayList<>();

        // Check position sizing
        Position largestPosition = getLargestPosition(userId);
        double positionRisk = calculatePositionRisk(largestPosition);

        if (positionRisk > 0.40) {
            insights.add(Insight.builder()
                .type("RISK_WARNING")
                .severity("HIGH")
                .title("High Concentration Risk")
                .message("You have 40%+ capital in one trade. Consider diversification.")
                .actionable(true)
                .build());
        }

        // Check daily loss limit
        double todayLoss = calculateTodayPnL(userId);
        if (todayLoss < -5000) {
            insights.add(Insight.builder()
                .type("LOSS_LIMIT")
                .severity("CRITICAL")
                .title("Daily Loss Limit Approaching")
                .message("You've lost ₹5000 today. Consider taking a break.")
                .suggestedAction("ENABLE_KILL_SWITCH")
                .build());
        }

        return insights;
    }
}
```

2. Behavioral Pattern Detection

java

```

public List<Insight> detectBehavioralPatterns(UUID userId) {
    List<Insight> insights = new ArrayList<>();

    // Overtrading detection
    int todayTrades = getTodayTradeCount(userId);
    if (todayTrades > 20) {
        insights.add(Insight.builder()
            .type("OVERTRADING")
            .message("You've made 20+ trades today. This often leads to losses.")
            .statistics(Map.of(
                "today_trades", todayTrades,
                "avg_trades", getAvgDailyTrades(userId)
            ))
            .build());
    }

    // Revenge trading detection
    if (detectRevengeTradingPattern(userId)) {
        insights.add(Insight.builder()
            .type("REVENGE_TRADING")
            .severity("HIGH")
            .message("Detected rapid trades after losses. Take a break.")
            .educationalLink("/learn/emotional-trading")
            .build());
    }

    // Holding time analysis
    double avgHoldTime = calculateAvgHoldingTime(userId);
    if (avgHoldTime < 5) { // Less than 5 minutes
        insights.add(Insight.builder()
            .type("SHORT_HOLDING")
            .message("You're holding positions for only " + avgHoldTime + " minutes on average.")
            .tip("Consider swing trading for better risk/reward.")
            .build());
    }

    return insights;
}

private boolean detectRevengeTradingPattern(UUID userId) {
    // Find recent loss followed by rapid trades
    List<Trade> recentTrades = getLastNTrades(userId, 10);

    for (int i = 0; i < recentTrades.size() - 1; i++) {
        Trade t1 = recentTrades.get(i);
        Trade t2 = recentTrades.get(i + 1);
    }
}

```

```

if (t1.getPnl() < 0) {
    long minutesBetween = ChronoUnit.MINUTES.between(
        t1.getExecutedAt(), t2.getExecutedAt()
    );

    if (minutesBetween < 2) {
        return true; // Rapid trade after loss
    }
}

return false;
}

```

3. Performance Pattern Analysis

```

java

public List<Insight> analyzePerformancePatterns(UUID userId) {
    // Time-of-day analysis
    Map<Integer, Double> hourlyPnL = groupTradesByHour(userId);

    int bestHour = hourlyPnL.entrySet().stream()
        .max(Map.Entry.comparingByValue())
        .get().getKey();

    int worstHour = hourlyPnL.entrySet().stream()
        .min(Map.Entry.comparingByValue())
        .get().getKey();

    return List.of(
        Insight.builder()
            .type("TIME_PATTERN")
            .message("Your best trading hour is " + bestHour + ":00")
            .statistics(hourlyPnL)
            .build(),
        Insight.builder()
            .type("AVOID_TIME")
            .message("You consistently lose money at " + worstHour + ":00")
            .suggestion("Avoid trading during this hour")
            .build()
    );
}

```

4. Kill Switch System

java

@Service

```
public class KillSwitchService {

    public void evaluateAndTrigger(UUID userId) {
        KillSwitch userSwitch = killSwitchRepo.findByUserId(userId);

        // Auto-trigger conditions
        double todayLoss = calculateTodayPnL(userId);
        int consecutiveLosses = getConsecutiveLosses(userId);

        if (todayLoss < userSwitch.getLossLimit() ||
            consecutiveLosses >= 5) {

            activateKillSwitch(userId, "AUTO_TRIGGERED");

            notificationService.send(userId,
                "Kill Switch Activated",
                "Trading paused to protect your capital. Resume tomorrow.");
        }
    }

    public void activateKillSwitch(UUID userId, String reason) {
        killSwitchRepo.updateStatus(userId, true, reason);

        // Cancel all pending orders
        orderService.cancelAllOrders(userId);

        // Set cooldown period
        LocalDateTime resumeAt = LocalDateTime.now().plusHours(24);
        killSwitchRepo.updateResumeTime(userId, resumeAt);
    }
}
```

Scoring System:

java

```
public int calculateDisciplineScore(UUID userId) {  
    int score = 100;  
  
    // Deductions  
    score -= getOvertradingPenalty(userId);  
    score -= getLossStreakPenalty(userId);  
    score -= getRiskExposurePenalty(userId);  
    score -= getEmotionalTradingPenalty(userId);  
  
    // Bonuses  
    score += getConsistencyBonus(userId);  
    score += getRiskManagementBonus(userId);  
  
    return Math.max(0, Math.min(100, score));  
}
```

2.6 MODULE 6 — Investment Universe

Features:

- IPO subscriptions (simulated)
- SIP (Systematic Investment Plans)
- Mutual funds
- ETFs
- Bonds

2.7 MODULE 7 — Education Layer

Features:

- Lessons on trading concepts
- Contextual help during trades
- Quizzes with rewards
- Gamified learning paths

2.8 MODULE 8 — Community

Features:

- Strategy sharing
 - Public leaderboards
 - Discussion forums
-

2.9 MODULE 9 — Admin Dashboard

Features:

- Asset management
 - User monitoring
 - System metrics
 - Market event triggers
 - Feature flags
-

3. API Design

3.1 RESTful Endpoints

Authentication:

POST /api/auth/register

POST /api/auth/login

POST /api/auth/logout

Trading:

POST /api/orders

GET /api/orders

DELETE /api/orders/:id

GET /api/positions

GET /api/holdings

Market Data:

GET /api/assets

GET /api/assets/:id

GET /api/assets/:id/history

GET /api/sectors

Wallet:

GET /api/wallet

POST /api/wallet/credits/purchase

GET /api/transactions

Analytics:

GET /api/analytics/summary

GET /api/analytics/trades

GET /api/reports/pnl

Insights:

GET /api/insights

GET /api/insights/risk

POST /api/killswitch/activate

3.2 WebSocket Endpoints

WS /ws/prices/:assetId - Live price updates

WS /ws/portfolio - Portfolio value updates

WS /ws/notifications - Real-time alerts

4. Frontend Architecture

4.1 Page Structure

/ - Landing page

/dashboard - User dashboard

/markets - Market watch

/trade/:symbol - Trading terminal

/portfolio - Holdings & positions

/orders - Order history

/wallet - Wallet & credits

/analytics - Performance analytics

/risk-center - Risk tools

/learn - Education

/admin - Admin panel

4.2 State Management

javascript

```
// Context structure
```

```
{  
  auth: { user, token },  
  market: { assets, prices },  
  portfolio: { positions, holdings },  
  wallet: { balance, credits },  
  insights: { recommendations, alerts }  
}
```

5. Deployment Strategy

5.1 Development Phases

V1 (Week 1-2): Core Trading

- Basic buy/sell
- Simple price simulation
- Wallet system

V2 (Week 3-4): Market Engine

- 100 fake companies
- Historical data
- Live updates

V3 (Week 5-6): Analytics

- Trade history
- P&L tracking
- Basic metrics

V4 (Week 7-8): Behavioral Coach

- Risk recommendations
- Pattern detection
- Kill switch

V5 (Week 9-10): Monetization

- Credit system
- Premium features

V6 (Week 11-12): Education

- Learning modules
- Contextual help

V7 (Week 13+): Production

- Microservices migration
 - Advanced features
 - Community
-

6. Security Considerations

- JWT-based authentication
 - Rate limiting on APIs
 - SQL injection prevention (Prepared statements)
 - CSRF protection
 - Input validation
 - Secure password hashing (BCrypt)
 - HTTPS only
-

7. Performance Optimization

- Database indexing on frequently queried columns
 - Redis caching for market data
 - WebSocket for real-time updates (reduce polling)
 - Pagination for large datasets
 - Background jobs for analytics
 - CDN for static assets
-

8. Monitoring & Observability

- Application metrics (Prometheus)
- Error tracking (Sentry)
- Logging (ELK Stack)

- Uptime monitoring
 - Database performance monitoring
-

9. Future Enhancements

- Mobile app (React Native)
 - Advanced charting tools
 - Social trading features
 - AI-powered insights (after 10k users)
 - Multi-asset support (Forex, Commodities)
 - API for third-party integrations
-

10. Success Metrics

User Engagement:

- Daily Active Users (DAU)
- Trades per user per day
- Session duration

Learning Outcomes:

- Lessons completed
- Win rate improvement over time
- Risk score improvement

Business Metrics:

- Credit conversion rate
 - User retention (7-day, 30-day)
 - Churn rate
-

Conclusion

This platform is a **portfolio-grade, startup-level system** designed to teach trading through experiential learning. The two core differentiators are:

1. **Realistic Market Simulation** - Multi-factor price models create believable market behavior
2. **Behavioral Coaching** - Rule-based insights that educate, not predict

By building in phases and focusing on these two pillars, you create a professional trading education tool that stands out in the market.